Cohomological Typing for Absolutely Everything

A DEPPY monograph on sheaf-descent, hybrid dependent types, trust, and generation

Repository-integrated draft

March 20, 2026

Programs are covered by local observations; types are the sections attached to those observations; proofs, semantic judgments, and generated artifacts succeed when the sections glue.

Abstract

This monograph takes the strongest idea running through DEPPY and pushes it to its logical limit: if typing is fundamentally a local-to-global problem, then the natural language for typing is not a flat set of judgments but a sheaf-like organization of local sections, overlap maps, trust labels, proof objects, and executable artifacts. The repository already contains several versions of this idea. The foundational `src/deppy/types/` hierarchy presents a sheaf-descent semantic typing substrate for Python. The `src/deppy/hybrid/` tree extends that substrate into a layered object whose coordinates are INTENT, STRUCTURAL, SEMANTIC, PROOF, and CODE, equipped with a trust lattice and an oracle monad. The equivalence machinery, anti-hallucination pipeline, theory packs, type expansion modules, zero-change analyzers, and prompt-driven agent all reuse the same local-to-global intuition.

The goal of this document is to describe that entire ecosystem as one research program: *cohomological typing for absolutely everything.* Here "absolutely everything" means not that every software question reduces to a single trivial theorem, but that the same geometric and dependent-typing viewpoint can organize foundational types, verification obligations, solver fragments, semantic oracle judgments, library theories, interprocedural transport, program equivalence, hallucination detection, prompt-driven planning, and even domain ideation. The manuscript therefore reads the repository as a single layered system rather than as a bag of isolated prototypes.

The central claims are straightforward. First, the plain type system in `src/deppy/types/` is still the object language: it gives DEPPY its carrier hierarchy, refinement types, dependent functions and pairs, equality-like objects, indexed families, and subtype obligations. Second, the hybrid system does not replace that language; it lifts it into a richer semantic object

$$\mathsf{HybridTy} : (\mathsf{Site}(P) \times \mathsf{Layer})^{\mathrm{op}} \to \mathsf{Type} \times \mathsf{Trust},$$

where coherence is measured by the vanishing of first Čech cohomology and semantic predicates live inside the oracle monad

$$T_O(A) = A \times \mathsf{TrustLevel} \times \mathsf{Confidence} \times \mathsf{Provenance}.$$

Third, many subsystems that appear separate in practice become legible as specializations of the same architecture: bugs as non-gluing cocycles, equivalence as gluable local isomorphism, theory packs as local solver sheaves, mixed-mode syntax as a user-facing bridge into plain obligations and hybrid contracts, and the generation agent as a typed cover-growing machine.

The document is deliberately repository-specific. It names the relevant files, acknowledges the places where the repository duplicates concepts or shifts vocabulary, and treats current examples—including trading, zero-change verification, and anti-hallucination—as evidence for a broader thesis rather than as disconnected demos.

Contents

I Foundations 1

1 Introduction 2

2 Scope and Thesis 5

3 The Cohomological Type Stack 7

4 Sites, Presheaves, and Locality 11

5 The Plain Type System 13

II Hybrid Evidence 15

6 The Hybrid Type System 16

7 Cohomology and Obstruction Theory 19

8 Trust, Oracles, and Proofs 22

9 Obstructions, Equivalence, and Hallucination 24

10 Theory Packs and Local Solver Geometries 26

III Elaboration and Synthesis 28

11 Type Expansion, Ideation, and Agents 29

12 Native Python, Zero-Change Checking, and Mixed Mode 32

13 Examples, Trading, and Repository Integration 34

14 Functorial Semantics: Rendering, Transport, and Synthesis 36

15 Implementation Program 39

16 Conclusion 41

A Appendix: Candidate Definitions and Results 43

B Appendix: Repository Map **44**

Part I

Foundations

Chapter 1

Introduction

DEPPY is easiest to misunderstand when its different subsystems are read as separate one-off experiments: a plain type hierarchy here, a hybrid dependent type story there, a cohomological bug-finding paper somewhere else, and an agent-driven generation runtime on top. The working thesis of this manuscript is that those pieces are better read as one architecture. The architecture begins from a simple observation: software understanding is local-to-global. We learn facts at boundaries, guards, call sites, proof obligations, library interfaces, and traces. We then try to compose those local facts into a global judgment about safety, meaning, equivalence, or implementation quality. When local facts agree, we obtain a global section. When they fail to agree, we do not merely have a vague error—we have a gluing obstruction.

That is why cohomological language keeps reappearing across the repository. In the older sheaf-oriented work, a program is analyzed through a site category of observation points, a semantic presheaf of local facts, and the computation of Čech cohomology to locate where those facts fail to glue. In the hybrid work, the same local-to-global idea is lifted to a five-layer product site where intent, structural facts, semantic judgments, proof artifacts, and code must agree. In the equivalence machinery, one asks whether local isomorphisms glue to a global one. In anti-hallucination, one asks whether a candidate artifact inhabits the expected type on both its structural and semantic faces. In prompt-driven generation, one grows a cover of typed module obligations and tries to glue them into a coherent project.

The phrase "cohomological typing for absolutely everything" is intentionally ambitious, but it is not meant as parody. It means that the same mathematical stance can organize far more of the repository than ordinary type-checking alone. Typing becomes a universal language of structured local evidence: value-level facts, control-flow facts, library facts, semantic judgments, proof obligations, and design intent all enter the same story. The right reaction is not to believe that every workflow literally computes a heavy cohomology group in the same way. The right reaction is to notice that the repository has already started treating the local-to-global pattern as its common semantic denominator. This monograph makes that denominator explicit.

It is worth saying more about what counts as a single project here. A manuscript can become long without becoming unified. It can merely accumulate examples, slogans, and modules. A foundational project earns unity only when its later constructions seem to have been latent in the earlier ones all along. The burden of this monograph is therefore not simply to explain many parts of the repository. It is to persuade the reader that once one grants the local-to-global viewpoint, the rest of the repository becomes a chain of increasingly rich consequences: first exact obligations, then layered evidence, then cohomological diagnosis, then trust-sensitive repair, then exploratory

elaboration, and finally agent-driven assembly. The paper should feel like a single river that keeps widening, not like a museum of related specimens.

The two most important distinctions in what follows are these. First, the plain type system in `src/deppy/types/` is not superseded by the hybrid system; it is the object language on which the hybrid system depends. It provides the type constructors, refinement syntax, binder operations, and subtype structure that later layers continue to use. Second, the hybrid system in `src/deppy/hybrid/` is not merely "the theorem prover side" or "the LLM side." It is a broader extension that adds verification layers, trust, provenance, semantic predicates, Lean translation, anti-hallucination, zero-change adoption, and prompt-driven planning. The hybrid story is therefore both more mathematical and more practical than a simple proof layer.

What this document is trying to do. The document has four goals. First, it gives a coherent account of the foundational plain type system and the layered hybrid extension. Second, it explains how site categories, presheaves, trust lattices, and dependent formers organize the repository's major ideas. Third, it shows how apparently distant components—theory packs, renderers, equivalence, ideation, generation agents, and trading examples—fit into the same architecture. Fourth, it tries to be honest about tensions in the repository: duplicated trust vocabularies, heuristic bridges from intent to formalization, and the difference between semantic acceptance and formal theorem proving.

Why the comparison to a foundational project matters. The aspiration behind the present document is not to mimic the vocabulary of better-known foundational programs, but to borrow one of their virtues: they make the reader feel that a few basic ideas change the subject all the way down. In that spirit, the early chapters here will insist on sites, sections, and exact obligations; the middle chapters will insist on layered evidence, trust, and cohomological obstruction; the later chapters will insist that generation, theory packs, rendering, and zero-change checking are not external applications but internal expressions of the same semantics. If the manuscript succeeds, the reader should feel that the repository is building one universe of discourse for software work, not merely a bag of clever techniques.

What it means to integrate the repository well. To integrate the repository well is not to mention every file exactly once. It is to show how the files cooperate semantically. A good paper should therefore explain why `src/deppy/types/refinement.py` matters to `src/deppy/hybrid/mixed_mode/compiler.py`, why `src/deppy/hybrid/core/types.py` matters to `agent/orchestrator.py`, why `src/deppy/equivalence/` and `src/deppy/render/` are easier to read after one understands gluing obstructions, and why the zero-change APIs in `deppy.hybrid` are not merely convenience wrappers but part of a deliberate adoption strategy.

A minimal slogan. If a single slogan is needed, it is this:

> *The plain type system gives* DEPPY *a language of exact semantic obligations. The hybrid system gives* DEPPY *a language of layered evidence about those obligations. Cohomology is the bookkeeping device that says whether the layers and local regions really glue.*

The remaining chapters expand that slogan until it can carry almost everything the repository is trying to do.

The reader is therefore invited to hold one question throughout the rest of the manuscript: emphif this really is one project, what stays invariant as the objects get richer? The answer offered

by the following chapters is that locality, transport, gluing, obstruction, repair, and explicit evidence remain invariant. Everything else is the elaboration demanded by software reality.

Chapter 2

Scope and Thesis

This chapter states the central thesis in a way that can absorb both the mathematical and engineering sides of the repository.

Definition 2.1 (Cohomological typing)**.** Cohomological typing is the discipline of representing typing information as local sections over a cover of program observations, transporting that information along restriction maps, and classifying global success or failure by whether those local sections glue. In the simplest case the data valued in each site is a type-level fact. In the richer cases used by DEPPY, the data also carries trust labels, provenance, semantic rubrics, proof obligations, and code artifacts.

The qualifier "for absolutely everything" can now be explained more carefully. It does not mean there is a single theorem that automatically decides all program-analysis problems. It means the repository repeatedly discovers the same shape of reasoning: local observations, overlap compatibility, gluing, obstruction, repair, and trust. That shape is visible in ordinary type checking, in bug diagnosis, in prompt-driven generation, in semantic oracles, in equivalence testing, in domain-specific theory packs, and in zero-change adoption for existing Python code. Once the common shape is visible, the phrase "typing" broadens. It comes to mean "typed local evidence attached to a site and moved through a gluing discipline."

The repository pushes this broadening along two axes. The first axis is expressivity. The plain type system already includes refinements, dependent function and pair formers, identity-like objects, indexed families, and subtyping. The second axis is evidence. Hybrid typing extends the foundational language so the same semantic object can be seen at the level of intent, structural constraints, semantic judgments, proofs, and code. The result is a repository in which types are no longer merely formulas over values; they are multi-layer objects that say what is claimed, how it was checked, how much it should be trusted, and where its remaining ambiguity lives.

A paper with this scope must also be careful about what it is *not* claiming. It is not claiming that every runtime execution in the repository passes through one uniform heavy symbolic cohomology engine. It is not claiming that oracle-mediated semantic judgments have the same epistemic standing as Lean proofs. It is not claiming that every module is equally mature. Instead, the claim is architectural: the repository is most coherent when those workflows are viewed as different instances of one local-to-global, site-aware, evidence-aware semantics.

The central thesis. The central thesis of this monograph is that the repository implements, in partial but already meaningful form, the following picture:

$$\mathsf{Sem} : \mathsf{Site}(P)^{\mathrm{op}} \to \mathsf{RefinementLattice}$$

for the foundational plain system, and

$$\mathsf{HybridTy} : (\mathsf{Site}(P) \times \mathsf{Layer})^{\mathrm{op}} \to \mathsf{RefinementLattice} \times \mathsf{TrustLattice}$$

for the hybrid system. The former provides the object language of obligations; the latter provides the layered evidence model in which those obligations are planned, checked, rendered, translated, discharged, and assembled.

Three design commitments. The thesis comes with three commitments that are visible throughout the code base.

1. **Native Python first.** The mixed-mode surface is designed so users can stay inside ordinary Python syntax and still benefit from rich semantics. Surface forms degrade gracefully before compilation.

2. **Structured evidence, not silent collapse.** Unknown or heuristic facts should remain visible as residual or lower-trust data. Trust levels and provenance are meant to prevent false certainty.

3. **Local-to-global semantics everywhere.** Bugs, equivalence, hallucination, library reasoning, and module planning are all interpreted as local facts that may or may not glue.

In short: the scope is large, but the architectural thesis is clean.

Chapter 3

The Cohomological Type Stack

A mature foundational project feels unified when its central ideas are not introduced as separate tricks for separate subproblems, but as a small vocabulary that keeps reappearing until the reader realizes that the whole enterprise is one object seen under several illuminations. That is one reason some mathematical projects feel inevitable rather than accidental. They persuade the reader that once a few basic constructions are granted, the later chapters are not bolt-ons but consequences. DEPPY should be read in that spirit.

The repository is easiest to trivialize when its components are described in isolation: there is a plain type system; there is a hybrid extension; there are trust levels; there is an oracle monad; there is a cohomological bug story; there are theory packs; there is a zero-change verifier; there is a generation agent. Read that way, the repository looks like a mixed research notebook. Read more carefully, it looks like a single stack whose levels share one grammar. The grammar is the grammar of local sections, transport, gluing, obstruction, repair, and eventual assembly.

The one-stack principle

The most useful way to say this is the *one-stack principle*:

> *Every major subsystem in* DEPPY *is a manifestation of one cohomological type stack. The lower levels supply the objects and local obligations; the middle levels supply the witnesses, trust annotations, and proof boundaries; the upper levels supply the methods by which typed local material is searched, transported, rendered, and assembled.*

The stack is not a metaphor pasted over arbitrary code. It is a practical account of how the repository is organized.

Definition 3.1 (The cohomological type stack)**.** The cohomological type stack is the five-level organization

$$\mathsf{Foundation} \to \mathsf{Object} \to \mathsf{Evidence} \to \mathsf{Semantics} \to \mathsf{Assembly}$$

in which:

1. **Foundation** supplies sites, covers, restriction maps, and presheaf structure.
2. **Object** supplies the exact type language of carriers, refinements, and dependent formers.
3. **Evidence** lifts those objects into hybrid witnesses distributed across intent, structure, semantics, proof, and code.

4. **Semantics** classifies and composes evidence via trust lattices, oracle effects, theorem translation, and residual obligations.
5. **Assembly** grows, repairs, and glues local artifacts into runnable or checkable systems.

The reward for naming the stack explicitly is that vocabulary starts to stabilize. A site is a place where one may speak locally. A section is typed local evidence. Transport is the movement of that evidence along a semantic map. Gluing is the passage from local evidence to a global artifact. Obstruction is non-gluing. Repair is a refinement of the cover, the local claim, or the evidence attached to the claim. The same words remain meaningful whether one is speaking about subtyping, hallucination, theory-pack discharge, or multi-module synthesis.

Five manifestations of one vocabulary

The repository's apparent diversity can now be restated as a controlled repetition of this vocabulary.

At the foundational level, one has sites and presheaves. A function is not seen as one flat blob of semantics but as a cover of boundaries, guards, intermediate values, call sites, and error-sensitive observations. That is already a local-to-global stance.

At the object level, one has the plain language in `src/deppy/types/`. Here the repository defines the actual symbolic objects that higher levels will manipulate: `TypeBase`, refinement types, Π-types, Σ-types, equality-like constructions, indexed families, and subtype obligations. This level says what must be true.

At the evidence level, one has the hybrid language in `src/deppy/hybrid/core/`. Here those same obligations acquire multiple manifestations: intent text, structural predicates, semantic rubrics, proof artifacts, and code. This level says how a claim is witnessed and how much confidence should be placed in the witness.

At the semantic level, one has trust, oracle effects, proof translation, discharge, and the bookkeeping of residual honesty. This level says what kind of evidence one has, what it is worth, and how it composes.

At the assembly level, one has type expansion, theory packs, zero-change checking, and the generation agent. This level says how local pieces are found, prioritized, and glued into a coherent system.

The central argument of this chapter is that these are not five separate research projects. They are five manifestations of one project.

Why the plain and hybrid systems belong to the same project

The plain and hybrid systems are sometimes described as though they belonged to different ages of the repository. This is historically understandable and mathematically misleading. The plain system is not an abandoned first draft; it is the exact object language that makes the hybrid system possible. The hybrid system is not a repudiation of the plain system; it is the evidence-bearing lift of the same obligations into a larger universe.

That relation can be stated succinctly. The plain system determines what a value, function, refinement, or witness *is*. The hybrid system determines what it means to possess multiple, potentially nonidentical, forms of evidence about that object and to compare those forms without erasing their differences. The hybrid system therefore presupposes the plain one in much the way a semantics of paths presupposes points.

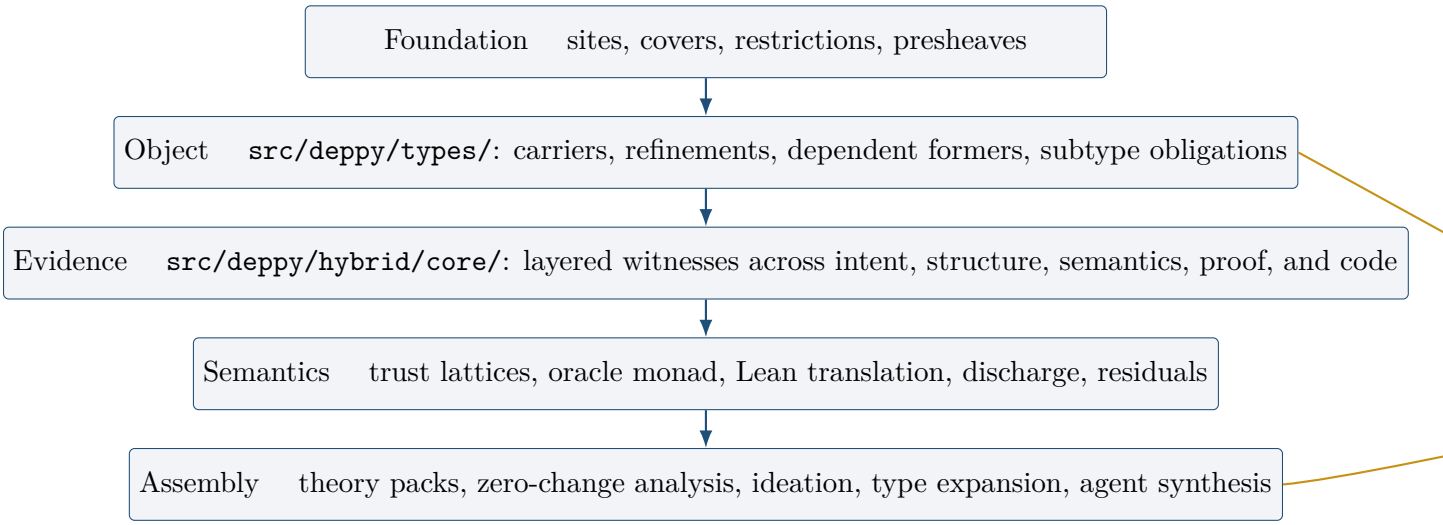

Figure 3.1: The cohomological type stack. The project becomes conceptually unified once the same local-to-global vocabulary is seen to govern each level.

One lexicon across engineering concerns

Once the stack is visible, many engineering decisions in the repository become easier to justify.

The mixed-mode syntax is no longer an ergonomic front-end awkwardly bolted onto a verifier. It becomes the public boundary at which ordinary Python enters the stack. Surface forms such as `@guarantee`, `assume()`, `check()`, and `hole()` are user-facing names for moves that later become sections, obligations, and proof targets.

Zero-change verification is no longer merely a convenience mode. It becomes the inverse boundary of the same project: instead of starting with explicit sections, the analyzer tries to infer local sections from existing code and to transport them upward into the stack.

Theory packs are no longer a plugin mechanism with independent semantics. They are local geometries inside the same project. Each pack says which sites matter, which solver or proof technology can act locally there, and what sort of trust upgrade is justified when the local problem is discharged.

The generation agent is no longer a free-floating LLM wrapper. It is a cover-growing and cover-repairing machine. Its task is to produce local sections whose interfaces, proof obligations, and semantics glue. Its failures are therefore not opaque generation mishaps; they are obstructions in the stack.

Axioms of the project

A project feels single when its commitments can be stated axiomatically. The repository is already near that point. A reasonably faithful list of axioms is the following.

1. **Locality first.** Software facts are obtained locally before they are claimed globally.

2. **Types before heuristics.** Every reusable judgment should be represented, when possible, as a typed local section rather than a nameless heuristic.

3. **Evidence should be explicit.** Trust levels, provenance, and residuals belong to the object, not merely to a log file.
4. **Non-gluing is structured.** Bugs, contradictions, hallucinations, and failed synthesis attempts should be classified as organized obstructions.
5. **Repair is refinement.** Failed globality should trigger stronger local models, finer covers, or better transports rather than silent fallback.
6. **Adoption must be gradual.** The stack should admit both zero-change analysis and deeply annotated mixed-mode programming.
7. **Generation belongs inside semantics.** Planning and synthesis are not beyond the theory; they are part of it.

This axiomatic register is one of the main things the monograph was previously missing. Once it is added, the later chapters stop sounding like catalog entries and start sounding like consequences.

The repository as one mathematical object

A final way to say the same thing is this: the repository is trying to build one mathematical object with many interfaces. One interface is foundational and symbolic. One is operational and proof-oriented. One is diagnostic and user-facing. One is heuristic and exploratory. What gives the project dignity is not that each interface is already perfect. It is that each interface can be described in terms of the same stack.

If the rest of the manuscript succeeds, the reader should reach the end feeling not that they have toured many ambitious ideas, but that they have spent time inside one ambitious idea that keeps differentiating itself into the forms demanded by software practice.

Chapter 4

Sites, Presheaves, and Locality

Long before one speaks about hybrid dependent types, one needs a language for local observation. The older sheaf-oriented work in the repository provides exactly that. A program is not treated as one monolithic proof obligation. Instead it is covered by observation sites: argument boundaries, branch guards, call results, error-sensitive locations, output boundaries, and in the more expansive monograph, also SSA values, wrapper boundaries, protocol or heap sites, proof sites, trace sites, and loop headers.

Definition 4.1 (Program site category)**.** Given a program P, a program site category $\mathsf{Site}(P)$ is a category whose objects are observation sites and whose morphisms are semantic transport maps induced by data flow, control flow, wrapping, projection, or proof transport. A cover of a region of the program is a family of sites whose joint information is intended to describe that region.

The point of a site category is not category-theory theatrics. It is an implementation stance. Once observations become sites, the analyzer can assign typed local sections to those sites and restriction maps to the arrows between them. For example, an argument site can carry a refinement about non-emptiness, a guard site can carry a branch-sensitive fact, a call-result site can carry a transported summary from another function, and an error site can carry a viability predicate describing when an operation is defined. The question "does the function type-check?" is replaced by the more informative question "do these local sections glue to a global contract?"

Definition 4.2 (Semantic presheaf)**.** A semantic presheaf on a program site category is a contravariant functor

$$\mathsf{Sem} : \mathsf{Site}(P)^{\mathrm{op}} \to \mathsf{Lat}$$

that assigns to each site a lattice of local sections and to each morphism a restriction map transporting information backward along the corresponding semantic dependency.

The repository repeatedly treats local sections as refinement-typed facts with enough structure to be merged, compared, and explained. This is already visible in the older paper material, where local sections are the atoms of bug finding and equivalence. It is also visible in newer code, where bridges and checkers keep trying to harvest and move site-local obligations.

Why the presheaf point of view matters. A presheaf gives DEPPY three important benefits.

1. It makes locality explicit. Every fact lives somewhere rather than vaguely on the whole program.

2. It makes transport explicit. Facts are moved along named restrictions rather than by invisible global heuristics.

3. It makes non-gluing diagnosable. Failures can be localized to overlaps and expressed as cocycles rather than as flat error lists.

This is the conceptual hinge between the older sheaf-cohomology work and the newer hybrid work. The hybrid system adds a layer dimension, but the idea of local sections and restriction maps is already present at the foundational level.

Running example. Consider a simple normalization function.

```
def normalize(values):
    total = sum(values)
    return [v / total for v in values]
```

A local section at the argument boundary may say "values is a list of integers." A local section at the division site may require "$total \neq 0$." A local section at a call site may transport assumptions from the caller. If those local sections do not agree on the overlap, the failure should be represented as a mismatch in the presheaf rather than as a mysterious one-line warning. That is the prototype of the cohomological story.

Remark 4.3. The repository sometimes writes as though the phrase "bugs are H^1 classes" is an entire theory by itself. The more exact statement is that the site/presheaf framework creates a mathematically principled place for gluing failures to live. The slogan is a consequence of that framework, not a substitute for it.

Chapter 5

The Plain Type System

The plain type system is the foundational language of DEPPY. Its home is `src/deppy/types/`, and its central role is to define the semantic objects that later passes manipulate. The file `src/deppy/types/base.py` opens by describing itself as a base hierarchy for DEPPY's sheaf-descent semantic typing system. That phrase is easy to overlook, but it says something important: the foundational system is already interpreted semantically and geometrically.

TypeBase and the carrier language

Every concrete plain type inherits from `TypeBase`. The abstract interface requires substitution, free-variable computation, structural traversal, equality, hashing, and a human-readable representation. Those are the operations one needs if types are to be symbolic objects in a real formal language rather than inert runtime labels.

The concrete hierarchy covers primitive Python types, top and bottom types, container-oriented types, callables, protocols, classes, and modules. This breadth matters because it means the plain system aims to be a semantic typing language for native Python, not a separate toy calculus with only theorem-prover examples.

Refinement types

The refinement layer in `src/deppy/types/refinement.py` defines the familiar shape

$$\{v : \tau \mid \varphi(v)\}$$

and then invests serious engineering effort in making predicates behave like real symbolic objects. Predicates support free-variable analysis, substitution, negation, conjunction, disjunction, implication, comparison, and named qualifiers. This is the first place where one sees the repository's long-term ambition: specifications are not meant to be English comments left floating around; they are intended to be structured semantic objects.

The crucial point for later chapters is that the plain refinement layer has no trust metadata. A refinement is simply a predicate-bearing type. There is no confidence score, no provenance, and no solver-versus-oracle distinction internal to the object itself. This absence is deliberate. It is one of the reasons the hybrid layer later becomes necessary.

Dependent formers

The file `src/deppy/types/dependent.py` gives the foundational system four of the most important ingredients for the whole repository:

- `PiType` for dependent functions $\Pi(x : A).B(x)$,
- `SigmaType` for dependent pairs $\Sigma(x : A).B(x)$,
- `IdentityType` for equality-like objects,
- indexed and quantified families for parameterized semantic structure.

These are not decorative imports from dependent type theory. They are what allow the rest of the repository to speak concretely about APIs whose outputs depend on inputs, witness-bearing results, transport of equalities, and theorem-carrying program fragments.

Definition 5.1 (Plain dependent function)**.** A plain dependent function type is a triple (x, A, B) represented in code as `PiType(param_name, param_type, return_type)`, with substitution and free-variable operations respecting binder hygiene.

Definition 5.2 (Plain dependent pair)**.** A plain dependent pair type is a triple (x, A, B) represented as `SigmaType(fst_name, fst_type, snd_type)`, where the second component type may depend on the first component.

Subtyping and obligations

The plain system does not stop at representation. The subtype checker in `src/deppy/types/subtyping.py` produces subtype answers and verification conditions. This is what turns the type hierarchy into an ordered refinement lattice rather than a mere syntax tree. When later hybrid code says it values types in a refinement lattice, this is the layer providing the foundational meaning of that phrase.

Why the plain system still matters after hybridization. It is tempting to think the hybrid system replaces the plain one. That reading is wrong. The mixed-mode compiler still lowers user-facing syntax into plain obligations such as refinements and Sigma-like witness forms. Lean translation still needs a foundational type-level language. The agent and checkers still benefit from a symbolic substrate with substitution and binder structure. The correct statement is therefore:

> The plain system is the object language; the hybrid system is a richer semantics for how objects of that language are layered, checked, and trusted.

Proposition 5.3. *If the hybrid system were removed but the plain type system retained,* DEPPY *would still have a meaningful dependent-and-refinement type language. If the plain system were removed but the hybrid packaging retained, the repository would lose the exact symbolic objects that many higher-level workflows require.*

The proof is architectural rather than formal: the code base repeatedly compiles into, imports from, or conceptually depends on the plain hierarchy.

Part II

Hybrid Evidence

Chapter 6

The Hybrid Type System

The hybrid system in `src/deppy/hybrid/` is best understood as a lift of the plain system into a layered, trust-bearing semantics. The key file is `src/deppy/hybrid/core/types.py`, whose opening documentation states the central mathematical object directly:

$$\mathsf{HybridTy} : (\mathsf{Site}(P) \times \mathsf{Layer})^{\mathrm{op}} \to \mathsf{RefinementLattice} \times \mathsf{TrustLattice}.$$

This formula does a large amount of conceptual work. It says that a hybrid type is not just a base type plus an extra annotation. It is a section defined over a product site whose second coordinate ranges over the ordered set

$$\text{INTENT} < \text{STRUCTURAL} < \text{SEMANTIC} < \text{PROOF} < \text{CODE}.$$

Layered sections

The hybrid core defines separate section-like structures for each layer. Intent sections store natural-language descriptions and structured claims. Structural sections store machine-checkable predicates, often tied to Z3 or runtime evaluation. Semantic sections store oracle-judged predicates with prompts, rubrics, and confidence thresholds. Proof sections store Lean-facing theorem content. Code sections store executable source and structural program representations.

The result is a model in which one and the same semantic object can be described at multiple levels without collapsing those levels into one another. This is the central advantage of the hybrid formulation. It gives DEPPY a disciplined answer to the problem of mixed evidence.

Definition 6.1 (Hybrid type)**.** A hybrid type is a tuple of layer sections

$$\tau = (\tau_I, \tau_S, \tau_M, \tau_P, \tau_C)$$

with components for intent, structural, semantic, proof, and code layers, together with restriction maps between adjacent layers and a trust label that records the strongest validated interpretation currently available.

Dependent formers survive the lift

The hybrid layer does not abandon the ordinary dependent type formers. Instead it defines hybrid analogues of them. The repository names `HybridPiType`, `HybridSigmaType`, and

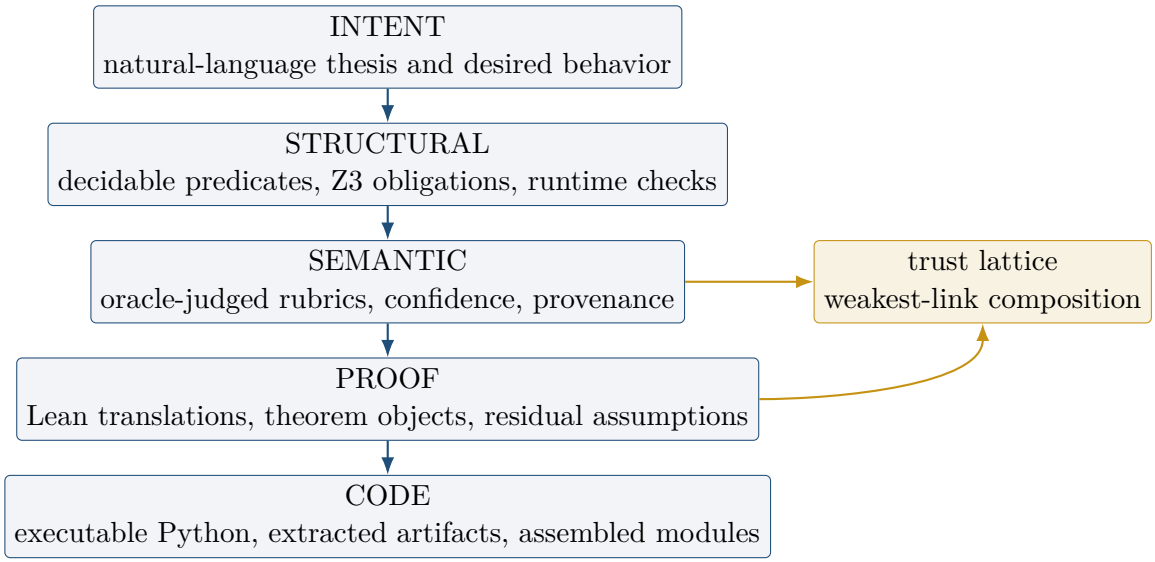

Figure 6.1: The layered object behind hybrid typing in DEPPY. The system keeps intent, structure, semantics, proof, and code distinct while forcing them to cohere under restriction and trust composition.

`HybridRefinementType`. Their point is not to replace Π and Σ with new syntax, but to reinterpret them in a setting where domain and codomain can themselves carry structural and semantic predicates, provenance, and trust labels.

This is why the hybrid system should be read as an extension rather than a fork. A hybrid Π-type is still fundamentally a dependent function. A hybrid Σ-type is still a witness-bearing or decomposition-bearing pair. A hybrid refinement type still says "values of this type satisfy this predicate," except that the predicate may now split into a decidable part and a semantic part:

$$\{x : T \mid \varphi_d(x) \wedge \varphi_s(x)\}.$$

Coherence as vanishing first cohomology

The hybrid core says a type is coherent when first cohomology vanishes:

$$\check{H}^1(\mathsf{Layer}, \mathsf{HybridTy}) = 0.$$

Operationally, this means adjacent layers agree on their overlaps. A function whose code runs but whose semantic layer rejects its behavior is not coherent. A proof artifact that postulates a theorem stronger than what the structural layer can justify is not coherent. A natural-language intent that is never realized by any code or proof is not coherent.

This criterion is philosophically valuable because it makes mismatch visible without pretending every mismatch is the same kind of failure. Some failures are structural. Some are semantic. Some are proof-level. The language of layered cocycles keeps those failures organized.

The hybrid geometry can also be pictured as a commutative square: restrictions in the site dimension should commute with forgetful transport between layers. For a local chart overlap $U \subseteq V$,

one wants a square of the form

$$\begin{tikzcd} \tau_S(V) \arrow[r, "\rho_{I\leftarrow S}"] \arrow[d, "\mathrm{res}_{U,V}"'] & \tau_I(V) \arrow[d, "\mathrm{res}_{U,V}"] \\ \tau_S(U) \arrow[r, "\rho_{I\leftarrow S}"] & \tau_I(U) \end{tikzcd}$$

to commute. When such squares fail to commute across a cover, the mismatch is precisely the sort of obstruction that the hybrid cohomology language is meant to expose.

Remark 6.2. The repository sometimes introduces convenience trust enums in one file and richer trust structures in another. A precise reading is that the hybrid package exposes a lightweight public face and a heavier core. The paper should acknowledge this rather than flatten it.

Chapter 7

Cohomology and Obstruction Theory

If one chapter must bear the weight of the title, it is this one. The phrase "cohomological typing" should not merely signal mathematical taste. It should explain why so many parts of the repository feel related. Cohomology is the device by which one records that local compatibility problems are not noise but structure. In this monograph, that means that the difference between success and failure is not exhausted by a boolean result. Success is the existence of a global section; failure is the presence of a cocycle that refuses to be resolved by the currently chosen cover.

From local sections to Čech complexes

Let a cover of the relevant program object be written as $\mathcal{U} = \{U_i\}_{i \in I}$, and let $\mathcal{F}$ be the presheaf of typed local evidence under discussion. Then the first terms of the Čech complex are

$$0 \to \check{C}^0(\mathcal{U}, \mathcal{F}) \xrightarrow{\delta^0} \check{C}^1(\mathcal{U}, \mathcal{F}) \xrightarrow{\delta^1} \check{C}^2(\mathcal{U}, \mathcal{F}) \to \cdots .$$

A 0-cochain is a family of local sections. Its coboundary measures disagreement on pairwise overlaps. A vanishing coboundary says that the family is compatible; a nontrivial class in $\check{H}^1(\mathcal{U}, \mathcal{F})$ says that the local data does not glue globally.

For the software situations emphasized in this repository, one often works with covers whose salient failures are already visible on pairwise overlaps. This is why first cohomology keeps reappearing. The point is not that higher cohomology is philosophically impossible. The point is that the repository's main engineering pathologies—mismatched contracts, inconsistent local semantics, ungluable modules, competing trust claims—already have a natural home in $\check{H}^1$.

Global sections, obstructions, and repair

The correct moral is therefore short:

> *A successful verification or synthesis episode is a vanishing first-obstruction claim relative to a chosen cover and chosen evidence presheaf.*

This sentence is powerful because it scales. For a plain typing problem, the cover may be a family of control-flow and data-flow sites, and the presheaf may record refinement obligations. For a hybrid coherence problem, the cover is enriched by a layer dimension and the presheaf now records both obligations and trust-bearing witnesses. For a synthesis problem, the cover is a family of local

module obligations. For anti-hallucination, the cover contains symbol-resolution, contract, and behavior sites. The outer mathematical sentence is unchanged.

Repair then becomes a map from obstruction classes to refined local structure. In words, repair is the act of changing the cover, strengthening a local predicate, refining a module boundary, or introducing a new witness so that the old cocycle becomes a coboundary. This is why the repository often treats diagnosis and repair as intimately connected. Once the obstruction has been localized, one already knows where a refinement should be attempted.

Five manifestations of first cohomology

The promise of a single project becomes most visible when the same cohomological sentence is instantiated in several apparently distant workflows.

Bugs. In the most classical case, a bug is a non-gluing local fact. A guard site may allow states that an arithmetic site forbids. A call site may assume a postcondition stronger than what the callee provides. A proof site may rely on a branch invariant that the code never really establishes. These disagreements live on overlaps and therefore naturally become 1-cocycles.

Hallucinations. A generated artifact can be structurally well formed yet fail to inhabit the semantic layer, or it can fail already at the structural layer because it names nonexistent symbols or grossly misstates a signature. In either case the failure is local-to-global noncoherence. The artifact does not inhabit the expected hybrid presheaf across the relevant cover. This is a cohomological failure, not merely a qualitative disappointment.

Equivalence failure. Two implementations may agree at many local sites and still fail to produce a global isomorphism. The equivalence machinery in `src/deppy/equivalence/` becomes much easier to understand once local semantic correspondences are read as candidate 0-cochains whose incompatibilities define a difference cocycle.

Generation failure. A module plan is a cover. Local code generation attempts to populate that cover with sections. If interfaces, assumptions, proof goals, or exported semantics fail to match across overlaps, one does not merely have "bad generation." One has an obstruction to descent. The agent is then in the business of refining covers and retrying gluing.

Trust inconsistency. Perhaps the most novel case is trust. One local path may deliver Z3-backed evidence, another only oracle-backed evidence, and a third may contradict both. These are not just numbers on a dashboard. They are typed local witnesses whose incompatibility can itself be treated as a non-gluing phenomenon. The language of trust-valued presheaves is therefore not optional if one wants the system to reason honestly about mixed evidence.

Obstruction mass and the quality of an elaboration

The repository's newer scalar-valued ideation layer is also easier to justify in this language. A candidate elaboration should not be judged only by thematic elegance or by an LLM's enthusiasm. It should also be penalized when it appears likely to generate unresolved obstruction mass. In the

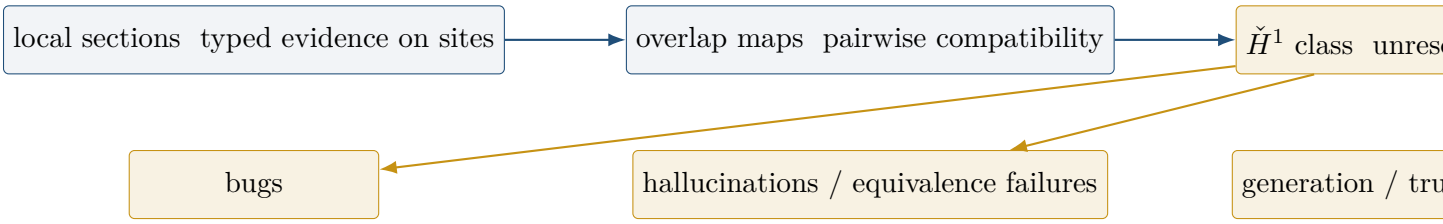

Figure 7.1: One cohomological sentence, many manifestations. The repository's main failure modes are organized by the same passage from local sections to overlap incompatibilities to first-obstruction classes.

current code, this appears as a penalty term related to the observed or estimated H^1-dimension of the ideation result. That move is mathematically more principled than it first appears. It says that elaborations carrying unresolved gluing risk should be scored lower even before full generation begins.

In other words, cohomology does double duty. It diagnoses explicit failures after local claims have been made, and it also helps rank prospective elaborations before the full cost of construction is paid. This is one of the clearest signs that the project's geometry is not ornamental. It actively shapes the agent's search.

Why first cohomology is not only about failure

There is a temptation to speak as though cohomology entered the story only when something has gone wrong. That reading is too narrow. The vanishing of first cohomology is not merely the absence of drama. It is the positive condition that local evidence has achieved global form. A gluable family of sections is itself a mathematical achievement. It is a certificate that multiple local accounts of a program agree. For a system like DEPPY, which deliberately keeps many kinds of evidence visible instead of collapsing them, such certificates are the closest thing to semantic harmony that one can ask for.

Repair as a research methodology

This suggests a wider philosophical point. The repository is not only trying to type and verify programs. It is trying to organize research methodology around the cycle

$$\text{observe locally} \to \text{compute incompatibility} \to \text{refine locally} \to \text{glue globally}.$$

That cycle is visible in program analysis, in theory-pack extension, in ideation, and in agent-driven synthesis. Once stated this way, it becomes plausible to claim that "cohomological typing for absolutely everything" is a methodology and not merely a slogan.

Proposition 7.1. *If a subsystem of software engineering can be represented by a cover of local sites, a notion of typed local evidence, and a repair loop that attempts to turn overlap failures into compatibilities, then that subsystem is in scope for the repository's cohomological typing program.*

The proof is not a theorem internal to one file; it is the architecture of the repository itself.

Chapter 8

Trust, Oracles, and Proofs

The hybrid system's most distinctive engineering move is to treat semantic judgment as a controlled effect rather than as an invisible side channel. This is expressed by the oracle monad

$$T_O(A) = A \times \mathsf{TrustLevel} \times \mathsf{Confidence} \times \mathsf{Provenance}.$$

A semantic check is therefore not merely a boolean returned by an LLM. It is a value with a trace: what model or process judged it, how confident that judgment is, whether it was cached, and which trust level should be attached to the result.

The trust lattice

The core trust ordering used by the hybrid types can be written as

CONTRADICTED < UNTRUSTED < RUNTIME_CHECKED < LLM_JUDGED < Z3_PROVEN < LEAN_V

Some convenience exports use slightly different public names, but the semantic pattern is the same: trust is ordered, and the order is used algebraically.

The meet operation records weakest-link composition,

$$a \sqcap b = \min(a, b),$$

while the join operation records strongest evidence,

$$a \sqcup b = \max(a, b).$$

These operations matter because they make compositionality honest. If one stage is only LLM-judged while a downstream stage is solver-proven, the composition should not suddenly pretend to be Lean-verified. The weakest link constrains the trust of the whole.

Structural versus semantic predicates

The oracle monad would be much less useful if every predicate lived inside it. The repository instead distinguishes structural predicates from semantic ones. Structural predicates are intended to be decidable or at least deterministically checkable. Semantic predicates encode claims whose meaningful evaluation may require richer interpretation, rubrics, or domain knowledge.

This split is the deepest reason the hybrid system can talk about anti-hallucination, solver discharge, and proofs in one breath without conflating them. Structural checks may be discharged by Z3 or deterministic evaluation. Semantic checks enter the oracle monad and produce confidence-bearing evidence. Proof translation may then reflect some of these facts into Lean, leaving others as assumptions or residual obligations.

Definition 8.1 (Hybrid refinement)**.** A hybrid refinement type is a base type T equipped with a structural predicate φ_d and an optional semantic predicate φ_s, written

$$\{x : T \mid \varphi_d(x) \wedge \varphi_s(x)\}.$$

The intent is that φ_d is decidable while φ_s is judged via the oracle pipeline and therefore carries trust and confidence metadata.

Lean translation and residual honesty

The repository's Lean translation and discharge cascade are significant because they preserve the distinction between theorem-backed and oracle-backed content. The discharge cascade typically moves through trivial discharge, Z3, oracle-assisted proof search, Lean checking, and residual fallback. This is an unusually honest design. It avoids the common temptation to treat all successful pipeline stages as equally formal.

A semantic predicate may be accepted at the semantic layer and even exported into a proof pipeline, but if it ultimately becomes an axiom or a residual assumption that boundary should remain visible. One of the strongest lessons from the repository is therefore that proof engineering and trust engineering must coexist. A system that hides its residuals is less trustworthy than one that openly classifies them.

Proposition 8.2. *Any semantics-preserving translation from hybrid types to Lean must either prove, postulate, or residualize every semantic predicate that lacks a purely structural proof.*

This proposition is conceptually obvious, but it is exactly the sort of obvious fact that software tools often ignore in practice. DEPPY does not ignore it; the hybrid design is built around it.

Chapter 9

Obstructions, Equivalence, and Hallucination

One of the repository's strongest unifying ideas is that several seemingly different problems become gluing problems once the right local sections are chosen.

Bugs as obstruction classes

In the sheaf-cohomological line of work, a bug is represented as a non-gluing local fact. A local section at one site may allow a state that another site's viability condition forbids. The disagreement lives on an overlap and can be collected into a 1-cocycle. When that cocycle is not a coboundary, it determines a nonzero class in first cohomology. This is the precise sense in which the repository says "bugs are H^1 classes."

Example 9.1 (Normalization bug)**.** Suppose an argument site allows an empty list while a downstream division site requires a nonzero sum. The call and error sites disagree on their overlap. The mismatch is a 1-cocycle; if no local strengthening already present in the cover resolves it, the class is nonzero and the bug is genuinely obstructive.

The slogan becomes especially valuable once it is quantified. The repository's older paper material argues that the rank of $\check{H}^1$ over $\mathbb{F}_2$ corresponds to the minimum number of independent fixes required to resolve the current family of gluing failures. Whether or not every implementation path computes that exact number in the same way, the architectural point is powerful: non-gluing is not just yes/no, it has structure and can be decomposed.

Equivalence as gluing of local isomorphisms

The equivalence machinery in `src/deppy/equivalence/` makes the same move for program comparison. Two programs are not compared by one magical global theorem from nowhere. Instead, one constructs local correspondences, local semantic differences, and overlap conditions. If the local isomorphisms glue, one obtains a global equivalence verdict. If they do not, one obtains a localized reason for failure.

This reuse of the same geometry is one of the best reasons to take the repository's mathematical self-description seriously. It would be much easier to implement bug finding and equivalence with

unrelated vocabularies. The decision not to do so shows that the local-to-global picture is doing real architectural work.

Hallucination as type inhabitation failure

Anti-hallucination is where the hybrid split between structural and semantic predicates becomes especially visible. A fabricated API is a structural failure: the relevant symbol, signature, or object does not exist. A semantic drift is a meaning failure: the code might be structurally well-formed while still failing the intended rubric or behavior. A contradiction between multiple checks is an inconsistency failure.

This is a very strong and very repository-specific claim. It says that hallucination should not be treated as mystical stochastic misbehavior. It should be treated as typed failure. The artifact has a required hybrid type. Structural checks attempt to inhabit the structural component. Semantic checks attempt to inhabit the semantic component. Proof and code layers contribute their own evidence. If the object fails to inhabit the required space, the failure can be named precisely.

Definition 9.2 (Hallucination as inhabitation failure)**.** Given an expected hybrid type T, a generated artifact a hallucinates when a fails to inhabit T at some required layer. Structural hallucinations fail structural inhabitation; semantic hallucinations fail semantic inhabitation; contradictory hallucinations produce incompatible evidence across layers.

This is one of the clearest examples of the paper's title. Once local-to-global typed structure is the common language, hallucination, equivalence, and bug finding are not unrelated downstream products. They are different ways for a typed section to succeed or fail to glue.

Chapter 10

Theory Packs and Local Solver Geometries

The repository does not attempt to solve every semantic question with one universal logic. Instead it provides domain-specific theory packs and theory-library machinery. This is another place where the cohomological typing viewpoint clarifies what the implementation is doing.

A theory pack is best understood as a local geometry of solvable questions. Tensor shape, nullability, boolean guards, witness transport, exceptions, heaps, protocols, loop invariants, and similar concerns each induce their own families of local sections and viability checks. The pack decides which sites are relevant, what local solver to invoke, which restrictions are legitimate, and how trust should be upgraded when a local discharge succeeds.

Why packs fit the site-based viewpoint

The older monograph on sheaf-descent semantic typing already pushes hard on the idea of site families and theory families. The hybrid and theory-pack code carries the same spirit forward. Rather than pretending there is one maximally expressive theorem prover inside the project, DEPPY distributes reasoning across packs. Each pack is responsible for a local stalk theory.

That phrasing is not decorative. A stalk theory answers local questions about a site. A shape theory can answer tensor-shape compatibility questions. A guard theory can answer branch-sensitive path-viability questions. A protocol or heap theory can answer ownership and field-availability questions. A witness transport theory can answer equality and transport questions. The global analyzer then composes these local answers.

Trust upgrades through local discharge

Theory packs also give a concrete answer to the question "where do trust upgrades come from?" They come from successful local discharge in a domain where the relevant solver or proof method is appropriate. A structural fact that begins as merely asserted may rise to runtime-checked or Z3-proven status after a local pack solves it. The hybrid trust lattice therefore does not float abstractly above the repository; it is fed by actual local reasoning modules.

Example 10.1 (Tensor theory)**.** A tensor-shape pack may certify that two tensors can be broadcast together, or that a reshape preserves total size. Those are local shape facts, but once transported

across call sites and composed with code and proof layers they become part of a global contract for a function. The same story generalizes to device compatibility, indexing safety, and order-sensitive tensor operations.

Absolutizing the title carefully

If the phrase "absolutely everything" is to remain honest, theory packs are part of the answer. The repository cannot yet reason deeply about every possible Python phenomenon. But it can repeatedly grow new local geometries and plug them into a common architecture. The title should therefore be read as a claim about the architecture's extensibility: any domain that can be given local sections, restrictions, and gluing conditions is in scope for cohomological typing.

Part III

Elaboration and Synthesis

Chapter 11

Type Expansion, Ideation, and Agents

The repository's prompt-driven agent is not an accidental extra on top of the typing system. It is one of the clearest demonstrations that the same semantic stance can govern planning and generation as well as checking.

Type expansion

The type-expansion modules in the hybrid package take a short natural-language description and elaborate it into a much richer plan: modules, covers, dependencies, obligations, interfaces, and eventually code-generation structure. This process is easiest to interpret as growth in a presheaf. The intent layer starts with sparse, natural-language local sections. Structural, semantic, proof, and code layers gradually acquire more content as the plan becomes more explicit.

The important conceptual shift is that generation is not described as free-form text completion followed by a verifier. It is described as typed elaboration. The prompt gives an intent section. Expansion grows a cover. Local generation produces candidate sections. Verification checks whether they glue. Repair loops try to resolve obstructions. Assembly produces a global artifact.

Ideation as typing

The ideation machinery is even more unusual. The repository includes a large mathematical ontology and domain-selection logic intended to enrich prompts by analogy to concepts from many mathematical areas. The right way to phrase this in the paper is not that the system has a magical creativity engine. It is that it tries to treat ideation itself as a typed search over richer local descriptions.

If a prompt about trading is enriched by analogy to algebraic geometry, order theory, signal processing, or topology, the resulting design search space is larger but also more structured. Ideation becomes a pre-typing phase that improves the quality of the later cover and the plausibility of the resulting local sections.

Scalar-valued typing

The ideation pipeline can be sharpened still further by introducing what the repository now calls *scalar-valued typing*. The idea is simple: before committing to a full elaboration, assign a scalar to the candidate elaboration that measures how strongly the current domain evidence supports it. In

implementation terms, `agent/cli.py` already tracks an LLM belief score, a mathematically-derived score based on validated analogies and certifiable structure, and per-area strengths. Scalar-valued typing packages these into a single typed quantity attached to the elaboration.

If E is a type elaboration and d is a target domain, then the scalar-valued typing score can be written schematically as

$$\sigma(E, d) = \alpha\, \sigma_{\mathrm{LLM}}(E, d) + (1 - \alpha)\, \sigma_{\mathrm{Math}}(E, d) + \varepsilon \max_i \sigma_i(E, d),$$

with $\alpha = 0.55$ and $\varepsilon = 0.05$ in the current implementation. The first term measures how strongly the language model believes the idea explains the problem in the chosen domain. The second term measures structural richness: validated analogies, deep connections, theorem schemas, certification targets, and penalties for unresolved H^1-mass. The final term lets especially strong local mathematical areas gently boost a globally promising elaboration.

This is a useful addition because it makes ideation behave less like unranked brainstorming and more like typed search. Scalar-valued typing does not replace ordinary type checking. Instead it orders candidate elaborations before the expensive downstream steps of full module expansion, proof planning, and code generation. The score is therefore neither a proof nor a mere UX flourish; it is a typed prioritization signal.

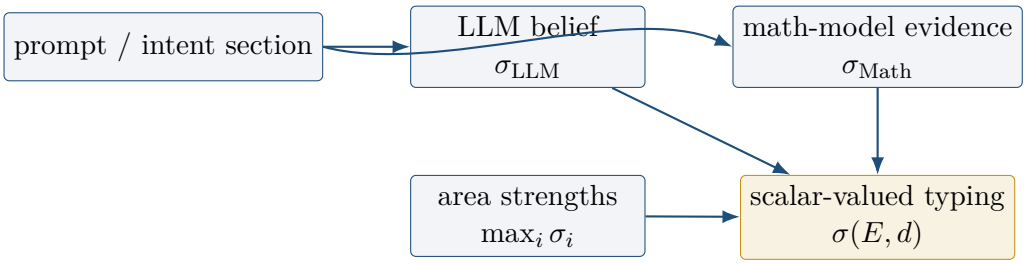

Figure 11.1: Scalar-valued typing adds a numeric layer over ideation: type elaborations are scored by domain fit before full expansion.

The orchestration agent

The agent orchestrator in `agent/orchestrator.py` embodies a multi-stage state machine: parsing, expanding, planning, generating, verifying, and assembling. The code is therefore easier to understand after one adopts the hybrid type picture. Each stage is not simply passing strings to the next. It is refining what kind of section exists, which layer it inhabits, and which trust-bearing obligations must be checked before the next stage is allowed to treat the result as global enough.

The importance of this loop is that it places generation inside the same semantics as checking. A paper that really integrates the repository should emphasize this strongly. In the current repository story, scalar-valued typing sits just before the expensive center of this loop, biasing the search toward elaborations whose local mathematical evidence most strongly supports the target domain.

Algorithm 1 Cohomological generation loop

```
Resolve user prompt into intent sections.
Expand intent into a typed cover of local module obligations.
for each local module site do
    Synthesize candidate code and local contracts.
    Check structural, semantic, and proof obligations.
    if an obstruction remains then
        refine the local section and retry
    end if
end for
Attempt to glue the verified local sections into a project.
Emit residual obstructions if gluing fails.
```

Chapter 12

Native Python, Zero-Change Checking, and Mixed Mode

One of the repository's recurring practical commitments is that users should not be forced into a theorem-prover syntax cliff before they receive useful results. This commitment has two implementation faces: zero-change checking for existing code and mixed-mode syntax for new code.

Zero-change verification

The package-level quick APIs and zero-change tools are designed so ordinary Python can be analyzed without being rewritten first. The repository's exports around `hybrid_check()`, `hybrid_analyze()`, and implicit presheaf extraction form a gentle entry point into the larger architecture. This is important because it shows that the hybrid system is not reserved for carefully annotated greenfield projects.

The right way to read zero-change verification is as inference of local sections from existing program structure. The analyzer extracts sites from source, docstrings, control flow, and library use, then runs structural and semantic passes over those inferred sites. Trust labels remain visible, so the user can distinguish inferred, solver-backed, and residual facts.

Mixed-mode syntax

The mixed-mode syntax in `src/deppy/hybrid/mixed_mode/syntax.py` is the converse adoption path. Instead of starting from bare existing code, one starts from ordinary Python augmented with controlled surface forms such as `hole()`, `spec()`, `@guarantee`, `assume()`, and `check()`. The crucial invariant named in the file is graceful degradation: before compilation the forms either pass through, return safe defaults, or raise a controlled placeholder exception. This allows editors, tests, and ordinary Python execution to remain useful.

```python
from deppy.hybrid import guarantee, assume, check, hole

@guarantee("returns a sorted list with no duplicates")
def unique_sorted(xs: list[int]) -> list[int]:
    assume("xs is finite")
    ys = hole("remove duplicates from xs")
    check("ys preserves the elements of xs")
```

```
    return sorted(ys)
```

The compiler path for these forms reveals the deep relation between the plain and hybrid systems. Surface forms compile into plain obligations such as refinements and Sigma-like witness objects, then re-enter the hybrid pipeline as trust-bearing contracts and proof obligations. This makes mixed mode the public handshake between the two type systems.

Why this matters for the paper's title

If cohomological typing is really "for absolutely everything," it must include both legacy programs and new proof-oriented code. Zero-change checking handles the first case; mixed mode handles the second. Together they show that the architecture is not only a research kernel but also a migration strategy.

Chapter 13

Examples, Trading, and Repository Integration

A monograph about repository integration should not stay at the level of definitions. It should show how those definitions illuminate the code that users actually see.

Examples as semantic stress tests

The examples shipped in the repository exercise several different faces of the system: proof-oriented examples, tensor and shape examples, mixed-mode examples, zero-change checking examples, and the trading examples that drive the prompt-based generation story. These are not merely demos. They are semantic stress tests for the thesis that one local to-global architecture can support multiple workflows.

Why the trading system matters

The trading story is useful because it forces the repository to coordinate many ideas at once. There is natural-language intent about discovering or constructing a strategy. There are structural constraints such as position sizing, leverage bounds, exposure rules, and interface boundaries between market data, strategy kernels, risk engines, execution, and portfolio state. There are semantic judgments about whether a strategy meaningfully realizes a stated thesis. There are proof ambitions around risk-critical invariants. And there is actual code generation and assembly.

The hybrid cover-oriented generation pipeline is therefore not theoretical ornament in the trading example. It is exactly the mechanism needed to keep multiple modules and multiple kinds of evidence from drifting apart. A trading project becomes a cover of local sections: market-data sections, signal sections, risk sections, execution sections, portfolio sections, compliance sections, and final assembly. The question "did generation succeed?" becomes a gluing question.

The newer scalar-valued typing layer fits naturally into this trading picture. Different mathematical lenses—for example stochastic calculus, financial mathematics, or control theory—can assign different scalar weights to the same type elaboration. Those weights do not prove the system correct, but they help the ideation stage decide which elaborations deserve expansion into the expensive multi-module cover.

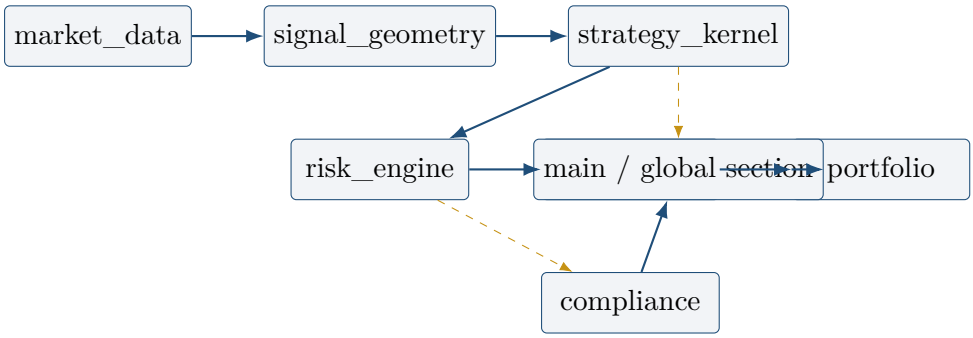

Figure 13.1: A repository-native trading cover. Solid arrows follow the execution and state flow; dashed arrows indicate proof and compliance constraints that must still glue into the final global section.

Repository-wide integration map

The repository can be read through the following semantic map.

- `src/deppy/types/`: the foundational object language.
- `src/deppy/hybrid/core/`: layered type semantics, trust, checker, and geometry.
- `src/deppy/hybrid/mixed_mode/`: user-facing bridge from Python syntax to obligations.
- `src/deppy/hybrid/discharge/` and `lean_translation/`: proof-facing boundary.
- `src/deppy/hybrid/type_expansion/`: planning and architectural elaboration.
- `src/deppy/hybrid/anti_hallucination/`: structural and semantic artifact validation.
- `src/deppy/equivalence/`: local-to-global comparison of programs and predicates.
- `src/deppy/render/`: turning typed and trust-bearing results into readable diagnostics.
- `src/deppy/theory_packs/` and theory libraries: modular local solver geometries.
- `agent/`: the orchestration loop that tries to grow a global section from prompt-local material.

Once this map is visible, the repository stops looking like a pile of unrelated prototypes. It looks like a large, uneven, but conceptually coherent attempt to turn local semantics into a universal organizing principle.

Chapter 14

Functorial Semantics: Rendering, Transport, and Synthesis

A unified project should not reserve its mathematics for only one corner of the system. If local-to-global semantics is truly the repository's common language, then even apparently secondary concerns such as rendering, transport, and synthesis should become legible in that language. This chapter argues that they do. The right conceptual move is to treat them as functorial constructions on typed presheaves rather than as utility layers sitting outside the theory.

Rendering as a natural transformation

Suppose $\mathcal{F}$ is a presheaf of typed local evidence. A renderer does not discover a new semantic fact ex nihilo. It reorganizes the existing typed content into another target language: English explanation, Lean code, diagrams, tables, or machine-readable reports. That suggests the right abstraction. Rendering is a natural transformation from a presheaf of typed sections into a presheaf of representations.

In the repository, this idea appears in multiple places. The web-facing explanations in `web/` are not external to the type story; they are human-readable images of it. The Lean translation pipeline in `src/deppy/hybrid/lean_translation/` is not just export; it is a semantics-preserving map from one typed universe into another. The report and rendering machinery in `src/deppy/render/` likewise maps trust-bearing objects into surfaces that a user or auditor can inspect.

What matters is that a good renderer respects restriction maps. If one moves from a whole function to a branch site or from a module to an overlap interface, the rendered object should continue to agree with the underlying semantics. That is exactly the sort of square one expects of a natural transformation.

$$\begin{tikzcd}
\mathcal{F}(V) \arrow[r, "R_V"] \arrow[d, "\mathrm{res}_{U,V}"'] & \mathcal{R}(V) \arrow[d, "\mathrm{res}_{U,V}"] \\
\mathcal{F}(U) \arrow[r, "R_U"] & \mathcal{R}(U)
\end{tikzcd}$$

Here $\mathcal{R}$ might be a Lean-syntax presheaf, a textual-report presheaf, or a diagram presheaf. The point is the same in every case: rendering should commute with locality.

Transport as restriction and extension

Interprocedural analysis and contract propagation are often discussed as if they were orthogonal to typing. In this project they are not. They are the mechanics by which typed information moves along a program's semantic edges. One may call that restriction, extension, transport, or pullback depending on which aspect one wants to emphasize, but it is still the same local-to-global geometry.

When a caller invokes a callee, one restricts attention from the caller's current state to the callee's input interface. When the callee returns, one transports postconditions back into the caller's context. When a wrapper preserves an invariant, one is witnessing a particular compatibility between the wrapped and unwrapped sites. When a zero-change analyzer infers a likely precondition from existing source, it is harvesting transportable local facts from ordinary Python and lifting them into typed form.

This is why the repository's adoption story remains coherent. The same semantics that govern explicitly annotated code can also govern inferred local structure. The difference is not one of basic ontology but of how much evidence is given up front.

Synthesis as cover elaboration

Perhaps the strongest test of unity is code synthesis. If synthesis sits outside the theory, then the project fractures: mathematics on one side, generation on the other. The repository is most interesting precisely because it refuses that fracture. The generation agent is best understood as an elaborator of covers.

A user prompt does not immediately produce a monolithic program. It produces intent-level material from which a cover is elaborated: modules, interfaces, invariants, proof targets, and gluing obligations. Local generation then attempts to produce sections on each chart. Checking passes test whether those sections agree on overlaps. Residual failures trigger cover refinement or local repair. Only at the end does one speak about a global artifact.

This account is far superior to saying that the repository simply "uses an LLM and then verifies the output." That description misses the architecture. The architecture says that synthesis itself is a typed local-to-global process.

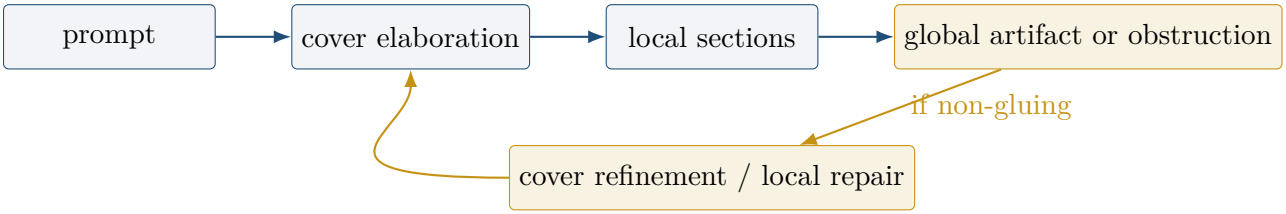

Figure 14.1: Synthesis as cover elaboration. The agent is most coherently understood as a machine for growing, testing, and refining covers until local sections glue.

Lean translation as a deep embedding

The Lean pipeline deserves special mention because it dramatizes the project's ambition. A lightweight renderer merely paraphrases typed content. Lean translation aims higher. It tries to

embed a portion of the repository's typed world into a theorem-proving environment whose kernel can then check part of the resulting content. This is not a casual format conversion. It is a deep embedding of selected structural and proof-bearing aspects of the stack.

That also explains why residual honesty matters so much. A functor into Lean cannot simply pretend that every semantic predicate has become a theorem. Some content becomes theorem statements. Some becomes lemmas with external evidence. Some becomes axioms. Some remains as residual assumptions. A faithful deep embedding preserves these distinctions rather than obscuring them.

Why this strengthens the sense of one project

The unifying power of the present viewpoint is easiest to appreciate negatively. Without it, rendering looks like documentation, transport looks like static-analysis plumbing, and synthesis looks like an attached generation system. With it, they become further instances of a single mathematical habit: typed local material should move coherently between sites, between layers, and between external representations.

That is exactly the kind of broad conceptual reuse a monograph needs if it is to feel like a single project rather than a collection of adjacent papers. The project is not merely about which types exist. It is about how typed local evidence circulates, is rendered, is upgraded, is repaired, and is finally assembled.

The repository as a functorial workshop

A final image may help. The repository is not only a checker and not only a generator. It is a workshop of functors and natural transformations built around typed presheaves. Some maps go downward into code. Some go upward into proofs. Some go sideways into reports, diagrams, and English explanations. Some shuttle information across call graphs and module boundaries. What makes these maps members of one family is that they are all expected to respect the same local-to-global semantics.

Once that expectation is made explicit, the repository reads more like a foundational research program. It is building not just tools, but a universe in which software artifacts, proof artifacts, semantic judgments, and generated designs can all be transported through a single coherent language.

Chapter 15

Implementation Program

This chapter turns the preceding conceptual story into a concrete implementation program for the repository as it exists now.

Principle 1: Keep the plain system as the stable substrate

The foundational hierarchy in `src/deppy/types/` should remain the canonical place for symbolic type-level structure: substitution, free-variable logic, refinements, dependent formers, indexed families, and subtyping. Hybrid features should continue to depend on that language rather than re-implementing its semantics in parallel.

Principle 2: Concentrate trust logic in a small number of core modules

The current repository exposes both lightweight and heavyweight trust vocabularies. This is not fatal, but it suggests an engineering direction: preserve public convenience exports, while concentrating the real semantics of trust, provenance, and evidence composition in a small number of clearly canonical modules. That will make papers, dashboards, and checkers less likely to drift apart.

Principle 3: Treat covers as first-class products

The older monographs already emphasize cover synthesis and site families. The generation agent and hybrid pipeline should continue to move in that direction. Rather than treating module plans or verification stages as opaque steps, they should make covers, overlap obligations, and gluing summaries first-class artifacts. This makes the local-to-global story inspectable by users and not only by implementers.

Principle 4: Make residuals visible

The paper has repeatedly insisted on epistemic honesty, and the implementation should do the same. Residual obligations, assumption boundaries, and heuristic semantic judgments should remain visible in manifests, renderers, and reports. The trust lattice is useful precisely because it avoids silent collapse.

Principle 5: Let theory packs grow the scope

The phrase "absolutely everything" should be realized by extensibility, not by claiming that every domain is already solved. New theory packs, domain bridges, and site families should remain the preferred path for broadening DEPPY's scope. The repository already has the architectural shape required for this growth.

Remark 15.1. The implementation program described here is conservative in one sense and radical in another. It is conservative because it preserves the repository's existing semantic core. It is radical because it proposes to explain more and more of the repository through a single local-to-global language rather than through disconnected subsystem-specific vocabularies.

Chapter 16

Conclusion

The repository now contains enough material to support a genuine monograph-level thesis. The thesis is not merely that sheaf theory is a pretty metaphor for program analysis, nor merely that hybrid dependent types are a convenient wrapper around solver checks and LLM judgments. The deeper thesis is that local-to-global semantics can be a universal organizing principle across a surprising amount of software engineering work.

The plain type system already provides a strong foundational language of exact semantic objects. The hybrid system already provides a layered semantics of trust-bearing evidence. The geometry modules already provide a language of gluing and obstruction. The equivalence modules already show how local isomorphism can become a global verdict. The anti- hallucination modules already show how artifact quality can be recast as inhabitation. Type expansion and the agent already show that planning and generation can be read as typed elaboration. Zero-change checking and mixed mode already show that the architecture can be adopted both from legacy Python and from proof-oriented new code.

What the revised manuscript has tried to add is not merely more detail but more inevitability. It has tried to show that these lines of work do not just coexist; they compel one another. Once one decides that software facts should be represented locally, that evidence should be typed, that differences in evidence quality should remain explicit, and that non-gluing should be treated as structure rather than noise, then a remarkable amount follows. One is led toward refinement lattices, toward hybrid witnesses, toward trust-sensitive semantics, toward cohomological diagnosis, toward theory packs, toward zero-change extraction, and toward synthesis as elaboration. The unity of the project lies not in a branding exercise, but in this chain of implication.

There is still real work ahead. The repository's different lines of development need to be made even more consistent. Trust vocabularies need consolidation. The bridge from intent to formalization remains heuristic and should be described as such. Covers, site families, and residuals should be made still more explicit in user-facing outputs. But those are exactly the kinds of tasks one expects in a large research prototype converging on a unifying architecture.

This is also why the phrase "absolutely everything" should be read with discipline rather than skepticism. It does not mean that every domain has already been mastered. It means that the project has identified an extensible semantic shape into which new domains can be fit. Whenever one can identify local sites, typed local evidence, transport maps, and a sensible notion of gluing, one has a candidate entrance into the same universe. The right way to extend the system is therefore not to bolt on one more isolated feature, but to ask what the local sections are, where their overlaps live, what trust structure they require, and how their obstructions should be repaired.

If the phrase “cohomological typing for absolutely everything” sounded excessive at the start, the point of this monograph is to make it sound less like excess and more like a research program. The program is this: whenever a subsystem of software work can be understood as local evidence that should glue globally, DEPPY asks whether typing, trust, and cohomology can be used to express that subsystem in one semantic language. Much of the repository is already an existence proof that the answer is often yes.

The deepest hope behind the project is therefore not just that particular theorems will be proved or that particular agents will generate better code. It is that software engineering can acquire a more stable conceptual center. In this monograph, that center is the claim that types, evidence, proofs, semantics, and generation are not unrelated vocabularies but progressive refinements of one local-to-global discipline. If the reader leaves with that impression, the document will have done what a foundational document should do: make later work feel less optional and more like the next chapter of the same idea.

Appendix A

Appendix: Candidate Definitions and Results

This appendix collects paper-ready definitions and results that fit the repository's current architecture.

Definition A.1 (Hybrid global section)**.** Let P be a program and let Layer be the ordered set INTENT $<$ STRUCTURAL $<$ SEMANTIC $<$ PROOF $<$ CODE. A hybrid global section for P is a section of the presheaf

$$\mathsf{HybridTy} : (\mathsf{Site}(P) \times \mathsf{Layer})^{\mathrm{op}} \to \mathsf{RefinementLattice} \times \mathsf{TrustLattice}$$

that is compatible under every adjacent restriction map.

Definition A.2 (Oracle-valued semantic predicate)**.** An oracle-valued semantic predicate on a carrier A is a map

$$\varphi_s : A \to T_O(\not\models)$$

where

$$T_O(B) = B \times \mathsf{TrustLevel} \times \mathsf{Confidence} \times \mathsf{Provenance}.$$

Theorem A.3 (Weakest-link trust composition)**.** *If semantic computations are composed by monadic bind in T_O, then the trust of the composite is bounded above by the meet of the trust levels of the constituents.*

Proof sketch. The monadic structure is defined to propagate trust via meet. Therefore every downstream semantic result inherits the weakest trust level seen along the path. This is exactly the engineering behavior desired by the repository, since high-trust stages should not erase low-trust inputs. □

Theorem A.4 (Obstruction criterion for hybrid coherence)**.** *If $\check{H}^1(\mathsf{Layer}, \mathsf{HybridTy}) = 0$, then every adjacent pair of layer sections glues to a coherent hybrid type over the chosen cover.*

Remark A.5. This theorem should always be read relative to a chosen cover and chosen restriction maps. The repository's mature position is strongest when it makes such relativity explicit.

Proposition A.6 (Mixed-mode compilation as a bridge)**.** *Every mixed-mode surface fragment can be understood as producing either a plain type-level obligation, a hybrid trust-bearing contract, or both. In particular, user-facing syntax is not a parallel semantics but a bridge into the existing object language and layered semantics.*

Appendix B

Appendix: Repository Map

Path or namespace	Role in the cohomological typing story
`src/deppy/types/base.py`	Foundational carrier hierarchy for plain typing.
`src/deppy/types/refinement.py`	Refinement predicates and the type-level shape of local facts.
`src/deppy/types/dependent.py`	Dependent function and pair formers, identity-like objects, indexed families.
`src/deppy/types/subtyping.py`	Ordering and obligation structure for the refinement lattice.
`src/deppy/hybrid/core/types.py`	Hybrid type semantics, layer order, trust-facing hybrid type formers.
`src/deppy/hybrid/core/checker.py`	Bidirectional hybrid checking and semantic-cache logic.
`src/deppy/hybrid/core/algebraic_geometry.py`	Layered presheaves, Cech complexes, cohomological obstruction machinery.
`src/deppy/hybrid/core/trust.py`	Trust lattice, evidence classification, provenance plumbing.
`src/deppy/hybrid/mixed_mode/syntax.py`	Gracefully degrading user-facing syntax for hybrid programming.
`src/deppy/hybrid/discharge/cascade.py`	Staged obligation discharge from trivial proofs to residuals.
`src/deppy/hybrid/lean_translation/`	Translation of typed and proof-bearing artifacts into Lean-facing forms.
`src/deppy/hybrid/type_expansion/`	Elaboration of prompt or intent material into typed plans and covers.
`src/deppy/hybrid/anti_hallucination/`	Artifact typing and structural / semantic hallucination classification.
`src/deppy/hybrid/zero_change/`	Inference over existing Python without annotation-first workflows.
`src/deppy/equivalence/`	Local-to-global equivalence, semantic diffing, and descent-style reasoning.

Path or namespace	Role in the cohomological typing story
`src/deppy/render/`	Turning typed results, residuals, and trust labels into readable surfaces.
`src/deppy/theory_packs/`	Domain-local solver geometries and library-facing semantics.
`agent/orchestrator.py`	Prompt-driven cover growth, generation, verification, and assembly.
`web/`	Human-facing explanation layer summarizing the mathematics and APIs.

This table should not be read as a dependency graph with one single topological order. It is a semantic map. The repository's pieces feed one another in multiple directions, but the map makes clear that the plain and hybrid type systems remain central reference points for most of the interesting work.